

ASSIGNMENT-2 MACHINE LEARNING

CHANDAN VARMA(G24AIT2017)

MANVENDRA PRATAP SINGH(G24AIT2163)

ALOK SINGH(G24AIT2021)

INDIAN INSTITUTE OF TECHNOLOGY-JODHPUR

Course Number - CSL7620

Instructor - DR.HARSHAL AKOLEKAR

Date *10-APRIL-2025*



PART - 1

Solution 1: dataset diabetes (1)

a) Find the optimum number of principal components for the features in the above-mentioned data.

- To reduce the number of features, I used PCA.
- Goal was to keep 95% variance.
- Plotted the explained variance, figured out how many components I needed and saved that in `optimal_components`.
-

function ⇒ *Solution 1.a (1.ipynb)*

b) Use any two regression models of your choice and find the prediction accuracy and error between the reduced data (with an optimum number of principal components) and the complete data.

Trained two models:

- Logistic Regression
- Random Forest

I tried both models on the full dataset (`X_scaled`) and the PCA version (`X_reduced`).

Then I checked accuracy and MAE and saved everything in a dictionary called `results`.

At the end, I compared both and found 'Random Forest' did better.

function ⇒ *Solution 1.b (1.ipynb)*

Solution 2: Fashion-MNIST dataset

a) Train the k-means model on f-MNIST data with k = 10 and 10 random 784-dimensional points (in input range) as initializations. Report the number of points in each cluster.

- Loaded the images, flattened 'em, normalized a bit.
- Ran k-means with 10 clusters starting from random spots.
- Then used `np.bincount()` to see how many images landed in each cluster. Pretty uneven in some runs.

function ⇒ *Solution 2.a (1.ipynb)*

b) Visualize the cluster centers of each cluster as 2D images of all clusters.

- Wanted to "see" what each cluster center looks like.

- So I reshaped the 1D center vectors back into 28x28 images and plotted them.
- Some looked like real clothes, some... not really.

function ⇒ *Solution 2.b (1.ipynb)*

c) Visualize 10 images corresponding to each cluster.

- Grouped images by cluster labels.
- Picked 10 random samples from each group and displayed them.
- Stored them in `cluster_images` dict just for better handling.
- Helped me guess what kinda stuff each cluster contains.

function ⇒ *Solution 2.C (2.ipynb)*

d) Train another k-means model with 10 images from each class as initializations , report the number of points in each cluster, and visualize the cluster centers.

- This time didn't start with random centroids —
- I picked one image from each real class to use as start points.
- Trained again and counted how many points per cluster like before.

function ⇒ *Solution 2.d (1.ipynb)*

e) Visualize 10 images corresponding to each cluster..

- Used `.inertia_` to get SSE from both models.
- Stored them as `sse_random` and `sse_class_based`.
- Lower SSE = better clustering.
- In my test, class-based usually gave lower error

function ⇒ *Solution 2.e (1.ipynb)*

f) Evaluate Clusters of part a and part d with Sum of Squared Error (SSE) method. Report the scores and comment on which case is a better clustering.

- Fills categorical columns with the most frequent value (mode).
- Fills numeric columns with their mean.

function ⇒ *Solution 2.f (1.ipynb)*

Solution 3: Implementation of Neural Networks

a) Load and preprocess the MNIST Digits Dataset.

- I grabbed the MNIST dataset (the digit one) using `fetch_openml()` which gave me all the images and their labels. I did some quick cleanup — mainly just scaled the pixel values between 0 and 1 (that's just good practice). Then I

split the whole thing into training, validation, and test sets. Nothing fancy here.

function ⇒ *Solution 3.a (1.ipynb)*

b) Implement a neural network with one input layer, one hidden layer, and one output layer using NumPy

- So this was kinda challenging. I made a basic neural network from scratch. Just one hidden layer — didn't wanna make it too deep or confusing.
- For weights, I went with Xavier initialization (seemed like a safe choice), and for activation, I stuck to sigmoid. Probably not the best but hey, it worked.
- Stored the activations in `A1`, `A2`, etc. Honestly had to debug a lot in this part lol.

function ⇒ *Solution 3.b (1.ipynb)*

c) Train the neural network with various hyperparameters (e.g., learning rate, number of hidden nodes)

- Used gradient descent (classic) with backpropagation to train the model.
- Tried a bunch of hyperparams before it kinda started making sense.
- Also had to add gradient clipping because without it the thing was blowing up.
- Logged losses and accuracies while training — dumped those into `train_loss`, `val_acc`, `train_acc`, and stuff like that.
- It took a while but eventually started improving.

function ⇒ *Solution 3.c (2.ipynb)*

d) Evaluate the performance of the neural network on the testing set

- Once it was trained, I tested it on the test set.
- Predictions came from the final layer and I used simple `np.mean()` to get the accuracy.
- Also added a confusion matrix (just to see where it messed up the most).
- It wasn't amazing, but definitely way better than chance — so that was a win for me

function ⇒ *Solution 3.d (1.ipynb)*

e) Implement the same neural network using sklearn and compare the results with the NumPy implementation.

- Then I was like... what if I let Sklearn do all the hard work?

- So I used `MLPClassifier` and just fed it the same training data.
- Didn't have to mess around with weights or gradients this time — it was super fast and worked well right out of the box.
- Logged the same stuff for this too, and compared it to my DIY NumPy version.

function ⇒ *Solution 3.e (1.ipynb)*

f) Plot the training and validation loss/accuracy curves (for both experiments).

- Finally, plotted both versions' accuracy and loss curves using matplotlib.
- Graphs made it super clear — the Sklearn model learned faster, but my NumPy one wasn't too far behind (after some tuning).
- Also helped spot when one of them was overfitting a bit.

function ⇒ *Solution 3.f (1.ipynb)*

PART - 2

Diabetes Health Indicators

Dataset: [Diabetes Health Indicators Dataset \(kaggle.com\)](https://www.kaggle.com/stone-island/diabetes-health-indicators-dataset)

1. Getting Familiar with the Data

1.1 Downloading the Dataset

I started off by grabbing the Diabetes Health Indicators dataset — used [kagglehub](https://www.kaggle.com/) to download it. Then, I just loaded it into a Pandas DataFrame using `pd.read_csv()`. I also saved a separate copy of the original data (`df_copy`) just so I could refer back if I mess up something during processing.

1.2 Exploring What's Inside

Before jumping into any cleaning or modeling, I wanted to see what I'm working with. So, I wrote a small function `explore_data()` — it gave me stuff like the shape of the data, column types, missing values, etc.

I also checked for duplicate rows using `df.duplicated()` — thankfully, didn't find any. And no null values either, which was a nice bonus.

1.3 Visual Checks and Plots

To get a better sense of the data, I made a few plots.

- First up, a **heatmap** to see how different features correlate with each other.

- Then, I looked at the distribution of the target column using a **bar plot** (just to make sure it's not completely imbalanced).
 - Finally, I created histograms for the numeric features. That helped spot any columns that were skewed or had weird outliers.
-

2. Prepping the Data for the Model

2.1 Missing Values

So, the first thing I checked was if any values were missing. Because if something's missing, it can mess up predictions later on. I had a function (`handle_missing_values()`) set up to fix this — it could either fill in the blanks with the average or remove columns that were mostly empty. But good news: no missing values were found, so I didn't have to run that part.

2.2 Duplicate Records

Next up, I looked for duplicate rows. They can throw off the training by repeating the same info. I used `remove_duplicates()` and let it clear them out if it found any. This helps avoid any bias caused by repeat data.

2.3 Outlier Detection

Some of the values looked way off from the rest — like too high or too low — so I figured they might be outliers. I used Z-score method (anything beyond 3 was considered out of range). The function `handle_outliers()` took care of removing them and left me with cleaner data.

2.4 Changing Categories to Numbers

Since ML models don't understand text, I had to convert any columns with text values into numbers. I ran `encode_categorical()` for that — it turned all the categories into numeric values using label encoding. After this, my dataset was all numbers.

2.5 Scaling the Features

Some columns had really large numbers, and others were small. To fix that and bring everything to the same level, I used standard scaling with a function called `scale_features()`. This makes training faster and more accurate.

2.6 Removing Similar Features

If two columns are way too similar, they don't help the model much. I used `select_features()` to drop features that had a high correlation (more than 0.85). That kept only the useful ones.

2.7 Fixing the Target Column

Lastly, I double-checked the target column. For this task, we needed only 0s and 1s — since it's binary classification. So I used `clean_target_variable()` just to be sure everything in that column was clean and usable.

3. Model Building

3.1 Data Splitting

First, I split the data into two parts — one for training the models and the other for testing how well they perform. I used an 80/20 split. So, 80% of the data is used for training, and the remaining 20% is for testing. I made a function `split_data()` to do this.

3.2 Picking the Models

I chose a few models to test things out — Logistic Regression, Random Forest, and XGBoost. These are common choices for classification problems. I used a helper function `select_models()` to set them up.

3.3 Training

Once the models were picked, I trained them using the training data. Nothing fancy here, just ran them using a function `train_models()` and stored the trained versions.

3.4 Checking Performance

After training, I evaluated each model on the test data to see how they actually perform. I used a bunch of metrics like accuracy, F1-score, and AUC to compare them. I wrote this in `evaluate_models()`.

3.5 Tuning

Lastly, I tried to improve the models by tuning some of their settings using `GridSearchCV`. This was done in `tune_hyperparameters()`. It took a while to run, but helped get better results in the end.

4. Model Deployment

4.1 Save the Best Model

Once I found the model that performed the best, I didn't wanna keep retraining it every time. Too much work. So, just saved it.

What I did:

- Wrote a small function called `save_model()`
- Picked the model that gave the best results
- Saved it into a file called `diabetes_model.pkl` — boom, done.

4.2 Load and Predict

Now that the model's saved, we gotta be sure we can load it back and actually use it.

Steps I followed:

- Made a `load_model()` function

- Loaded the `.pkl` file
- Passed in some new test data — and yep, it worked fine. Model gave predictions like expected.

4.3 Convert User Input

So yeah, doctors or users might type stuff in plain English or whatever form, but the model needs it neat and numeric.

What I did:

- Built a function `convert_doctor_input()` — this takes raw input from the doctor (like age, BMI, etc.)
- It reshapes that into a proper DataFrame, with all the right columns in the right order
- Then we pass that cleaned-up input to the model for prediction — and it gives back the result (Diabetic or not)

4.4 Frontend with Streamlit

We used **Streamlit** to build a simple UI where users like doctors or patients can enter details—age, height, weight, and a few symptoms. The app figures out **BMI** from height and weight on its own.

After clicking the "**Predict**" button, the info is passed to the model, and the prediction shows up—either **Non-Diabetic**, **Pre-Diabetic**, or **Diabetic**.

The app also supports language options (like Hindi), so it's easier for everyone to understand. Overall, Streamlit helped bring the model to life with a clean and usable frontend.